DLVSP Lab 1

Carlos Celaya and Marcel Hofmann

I. INTRODUCTION

This report concludes the work done on video classification with the UCF-101 (University of Central Florida 101) [1] dataset using various techniques. These three approaches have been used: Random class assignment, fixed class assignment, classification by fine-tuning an InceptionV3 network [2] and training an LSTM (Long Short-Term Memory) [3] in combination with a frozen InceptionV3 network. The goal is to find out which method is the best one to solve the video classification problem. Results show that in general the more complex approaches deliver better results. Still, the results depend on how much data (amount of classes for classification and the amount of videos for each class) and what kind of data was used (length of video).

Section 2 outlines the theoretical background of this report, mainly focusing on the CNN (Convolutional Neural Network) fine-tuning approach and the combination of a CNN with an LSTM. The third section highlights the implementation of the project, providing details about the classes and scripts used to obtain the results. A small introduction to the used dataset (UCF-101) is given in section 4. In section 5, the results of applying all algorithms are shown and analyzed. Finally, a conclusion of the project can be found in section 6.

II. METHOD

This assignment includes implementing the video classification methods using Python, especially Tensorflow [5]. The theory behind these methods is briefly explained in this part of the report.

A. Random Class Assignment

Having a video, a random class is assigned to it. Having this method, the classification by the system is not made via actually "having a look" at the video but just randomly guessing.

B. Fixed Class Assignment

This method works similar to the Random Class Assignment, but the class assigned is always the same one. Every video is labeled with the same class. A random class guess is performed to get a precision as high as possible.

C. Finetuning an InceptionV3 neural network

In contrast to the previous two approaches, this one is the first one that actually works with the data. Having an already pretrained InceptionV3 convolutional neural network that was trained with the ImageNet dataset, slight adaptations are performed to make it capable of classifying frames of videos. This method is called "Transfer Learning", [4] since knowledge of another training is transferred to a new domain. When setting up the InceptionV3 network, only the trained bottom and mid layers are pulled, the top layers (linear layers for classification) are replaced by untrained linear layers with the sizes needed for the classification task. A convolutional neural network using an inception approach actually runs multiple convolutions of different kernel sizes in one layer to capture patterns of different sizes, making it an effective approach for image classification. Unlike end-to-end sequence modeling approaches, this a single InceptionV3 network treats video classification as a frame-by-frame task: each video frame is classified independently. It does not know about other existing frames of the video.

The finetuning of this model is performed in two stages: The first stage consists of only training the top (linear) layers of the model. After a small set of epochs, only the second half of the model is trained (includes freezing the first half). Only the second half is trained because in later convolutional layers, models generally learn to search for patterns related to actual classes of the classification problem. Early convolutional layers are used to extract simpler patterns such as horizontal and vertical borders (appearing in mostly all images independently from the actual domain).

D. Classification with InceptionV3 and LSTM

The last approach is defined by using a pretrained InceptionV3 network for feature extraction. These features are used to train a newly initialized LSTM model. The InceptionV3 model is completely initialized (including top layers) but the actual extracted features are gained by the last average pooling layer of the network. The network does the feature extraction for all frames of the video, creating a sequence of frame features. This sequence of frame features is fed into the LSTM. Like this, a state is obtained encoding the information retrieved from all frames of the video. It makes this algorithm the only one using an entire video sequence to perform the classification. Since this is a many-to-one problem, only the last state of the LSTM is used for the classification. It is fed into a linear classification layer providing the probabilities for all video classes.

III. IMPLEMENTATION

In this section, the structure and implementation of the project is explained. The actual project can be divided into three parts: The random class assignments (session 1), the finetuning approach (session 2) and the CNN with LSTM approach (session 3).

A. Data Preparation

When having the raw project setup, the data needs to be shifted to the correct location in the project to make it usable for training. Extracting the zip file containing the last five classes, all videos land in the `data` folder. In later stages of the project, more classes are needed to fulfill the assignments, so the entire dataset is downloaded from the internet. It comes as a `.rar` directory. When unraring it, data is already sorted and thus is not in the format needed to execute the scripts for data preparation. Because of that, a new `move_files()` function has been implemented for moving data (cf. `1_move_files.py`). Actual video movement depends on the files in `data/ucfTrainTestlist`. The files in it tell where to move which video. Each video is included in the trainlist or the testlist. There are three possible combinations of creating train and test set with a train-test ratio of approx. 70:30. The data preparation consists of moving the files to train and test folder using `1_move_files.py` and extracting the actual video frames of each video using `2_extract_files.py`. The actual frame extraction is performed via `ffmpeg` [6]. In addition to the frame extraction, also a CSV (Comma-Separated Values) file is created with the following rows (one row = one video): train/test, class name, video name, amount of frames.

B. Dataset Class

For the UCF-101 dataset, a dataset class has been created to manage the large amount of data efficiently. The selection of data depends on setting the `seq_length` and `class_limit` property of the dataset. `class_limit` is used to define the maximum amount of classes included. When using the entire dataset, the list of classes used in the dataset instance needs to be flipped to select the last classes (to stick with the task). `seq_length` is used set a limit for frames of a video. Setting a low value of sequence length includes videos of small length. Setting a high value of sequence length includes videos of high length. Frames of the videos are retrieved using the `data_file.csv` (created during data prep) containing locations of videos belonging to the classes. There is the possibility to rescale a sequence of images to an amount of frames with length `seq_length`. Instead of using the frames, there is also an option to use the retrieved features of an image (by the InceptionV3 network for LSTM). Also, there is the possibility to not load all data into the memory at once but generate them during the training process (to save memory). The dataset class is created in every training script.

C. Random and Fixed Class Mode

The assignment of a random class and a fixed class is implemented into one file. By changing the string value of `fix_mode` to a class name occurring in the dataset (cf. `data/train` or `data/test`), this value is compared to the classes of all available videos. For the random mode, for each sample, a new class value is selected randomly in range of the number of classes. For both modes, the number of correct predictions is divided by the total amount of samples to get the accuracy.

D. Finetuning the InceptionV3 network

The neural network and the training function are implemented using Tensorflow in `train_cnn.py`. The `main()` function includes the model and generator (for adapting images randomly) creation. For training, there is the option to continue the training of a model (by setting a model path in `weights_file`) or start training a new model. For continuing training, only training the top layers is skipped. The model creation happens in the `get_model()` function. It creates the pretrained InceptionV3 architecture. The pre-trained model was loaded with `includetop=False`, removing the original classifier. Then, a Global Average Pooling layer, a linear layer with ReLU (Rectified Linear Unit) activation and a final linear layer with the number of classes as output with Softmax activation (for probability distribution) are added. There are two other functions: `freeze_all_but_top()` (for freezing all layers except last two linear ones) and `freeze_all_but_mid_and_top()`. The second function does the freezing from value `split_at=172` (function input param). Both functions do the (un-)freezing by looping through the model's layers. The first stage of the training is performed using RMSProp (Root Mean Square Propagation) to allow a rapid adaption of the last two layers to the classification problem. For training the bigger part of the network, Stochastic Gradient Descent is used since it is a bit more stable (also got small learning rate of 0.0001) training a bigger size of a network (avoid catastrophic loss). Categorical Cross-Entropy Loss is used to calculate the loss. The first training phase is 10 epochs to adapt the new dense layers. Afterward, the middle and top layers are fine-tuned for 100 epochs. Checkpoints, early stopping, and TensorBoard logging were implemented via Keras callbacks. Model checkpoints and logs saved under the `/data` directory. There is also a script used to visualize training progress (`plot_train_cnnlog.py`)and a script to validate the model by feeding in a set amount of `nb_images` and getting the sorted class predictions (`validate_cnn.py`).

E. InceptionV3 and LSTM

The combination of InceptionV3 and LSTM appears to be one big neural network, but in this project it is implemented in a two step approach: First extract frame features including saving them to `data/sequences`. Then train the LSTM separately. This makes sense because having the data saved, it is already available and the features do not have to be extracted every epoch again (do not change because the Extractor stays not finetuned). For the extraction, an `Extractor` class is created (`extractor.py`), incorporating the InceptionV3 network that outputs the values of the last average pooling layer. This class is used in `extract_features.py`, where a loop iterates through the rows (=videos) of the dataset (depending on set sequence length and class limit). Each video is cut into frames.

Some frames are left out depending on the wanted sequence length. All relevant frames are processed in the Extractor model. The sequence of features is saved. To make the extraction faster, a script using multiple processes is implemented in `extract_features_multi.py`. The second part is the training of the actual LSTM classifier. The class `ResearchModels` itself is defined in `models.py`. It consists of the LSTM unit returning only the last state, one linear layer with ReLU activation and dropout and one layer with the amount of classes as output and Softmax. The training is defined in the `train.py` script. As for the fine-tuning, it includes dataset creation, the possibility to keep all sequences in memory or generate them using a generator (to save memory) and callback implementation. Still, this time not the actual frames are used, but the extracted features of them saved at `data/sequences`.

F. Running the Code

For convenience, the scripts from all three sessions are combined into one directory, since there are dependencies on each other. To be able to perform trainings, the data needs to be preprocessed in the right location. For data preparation, the extracted data (from .zip or .rar) needs to be moved to the `data/` directory. Having selected the correct `move_files()` function, `1_move_files.py` and `2_extract_files.py` can be executed using the `python <filename>.py` command. For performing other tasks such as feature extraction or training, run the files using the same command but from the root directory. To be able to train the LSTM, feature extraction has to be performed previously with features in `data/sequences`. For completing the tasks, the parameters "sequence length" and "class limit" of the run need to changed. Their initialization can happen either at the beginning of training scripts or in the `main()` function of the script. For plotting the Tensorflow logs, the correct `.log` file name needs to be set as the value of `training_log` in the scripts for plotting.

IV. DATA

The UCF101 dataset was used, The dataset consists of 5,149 video clips distributed across 40 human action classes, divided into 3,688 training (71.6%) and 1,461 testing (28.4%) samples. Each video contains an average of 25.1 frames (median = 23.0, standard deviation = 12.1), ranging from 5 to 215 frames, indicating short, variable-length clips typical of human activity recognition datasets. The frame distribution is consistent between the training and test splits, suggesting balanced sampling without duration bias. The number of videos per class ranges from 100 to 166, demonstrating a relatively uniform class balance . Actions such as RockClimbingIndoor, PlayingSitar, and Rowing exhibit longer average durations, reflecting more complex motion patterns or recording conditions (1). For computational efficiency, subsets were extracted:

5 classes: Volleyball Spiking, Walking with Dog, Wall Pushups, Writing on Board, YoYo.

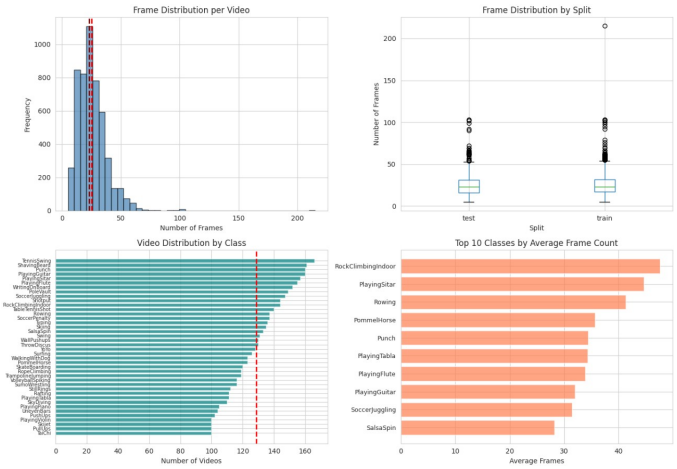


Fig. 1: Statistical analysis of the subset. In the first graph the frame distribution per video is representated. In the second graph the Frame distribution by split. In the third graph the video distribution by class. In the fourth graph the top 10 classes by average frame count

10 classes: From Tennis Swing to YoYo.
20 classes: From Skijet to YoYo.
40 classes: From Playing Violin to YoYo.

Each video of the dataset was divided into frames. The frames were resized to match the input shape of the chosen CNN model and the pixel values were normalized. During training, data augmentation was applied to increase variability. Two labeled frames can be seen in Fig. 2:

For the LSTM, the extracted features from the videos of UCF101 are used as training data.

V. RESULTS AND ANALYSIS

In this section, the results of the tested approaches are described and analyzed.

A. Random and Fixed Mode

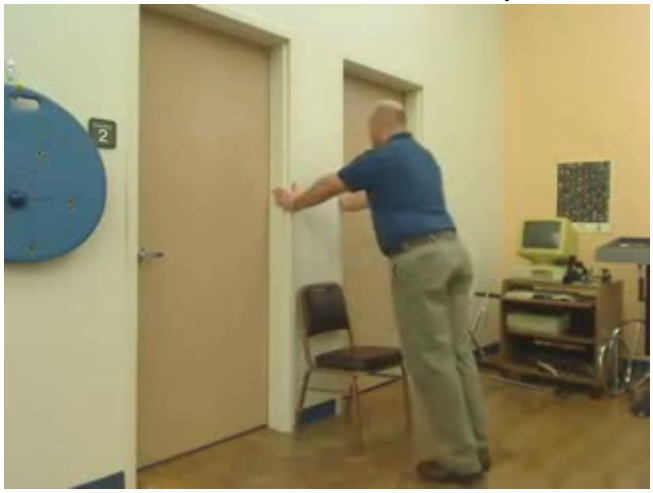
The random baseline yield an accuracy close to the theoretical chance level (approx. 20%). The results of the fixed mode results vary depending on which class was chosen, approximately matching the frequency of each class in the test set (Table 1). The fixed mode corresponding to the most represented class achieves the highest baseline accuracy, while the fixed mode aligns with the least frequent class that performs the worst. Performance differences reflect dataset imbalance rather than model capability.

Fix mode	Randomly Matched	Mode Matched
VolleyballSpiking	20.96%	17.87%
WalkingWithDog	17.10%	18.95%
WallPushups	20.96%	18.95%
WritingOnBoard	21.41%	23.42%
YoYo	20.80%	19.72%

TABLE I: Comparison of results for each activity with each fix mode



(a) Frame of video with label "Archery"



(b) Frame of video with label "WallPushups"

Fig. 2: Frames of two different videos with their labels

B. InceptionV3 finetuning

The model achieves excellent performance for smaller class subsets (5 and 10 classes), maintaining around 90% accuracy. However, as the number of classes increased, the model's accuracy decreased substantially.

When using transfer learning, the pretrained CNN provides strong generalization for a small set of target classes because many of the learned features (edges, textures, body poses) transfer effectively. However, with more classes, intra-class variability grows and inter-class differences become subtler, making the classification task more challenging. Since only a limited portion of the CNN is fine-tuned, the model could not fully adapt to the increased complexity.

Having a look at the training curves both training and validation accuracy curves converge quickly with minimal overfitting as it can be seen in Figs. 3 and 4.

For 20 and 40 classes (Fig. 5 and 6), both models experience a reduction in terms of accuracy. However, no overfitting is experienced since both curves are very close. So, it is deduced that if more epochs were applied, the results could be improved.

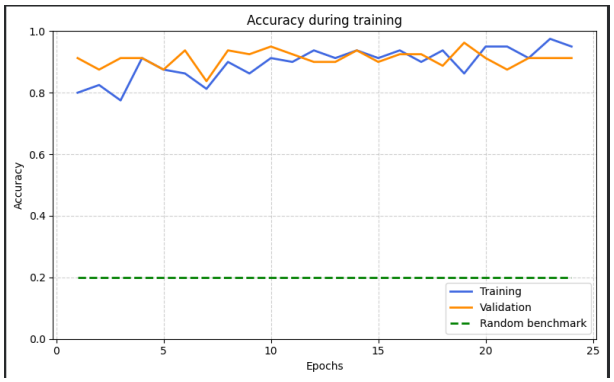


Fig. 3: CNN fine-tuning accuracy with `class_limit = 5`



Fig. 4: CNN fine-tuning accuracy with `class_limit = 10`

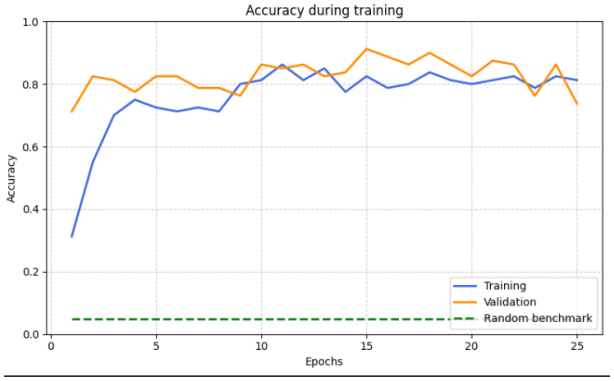


Fig. 5: CNN fine-tuning accuracy with `class_limit = 20`

The finetuning itself is pretty fast since only parts of a bigger model are adapted. This makes it suitable for fast tests and when not enough memory is available.

C. InceptionV3 and LSTM Combination

Using the approach with InceptionV3 and LSTM, accuracies above 90% can be achieved (Fig. 7). The model is better than the InceptionV3-only approach. Still, the success strongly depends the sequence length of the sequences. Looking at the plot, also the data selection when creating the dataset instance plays a huge role. With a sequence length

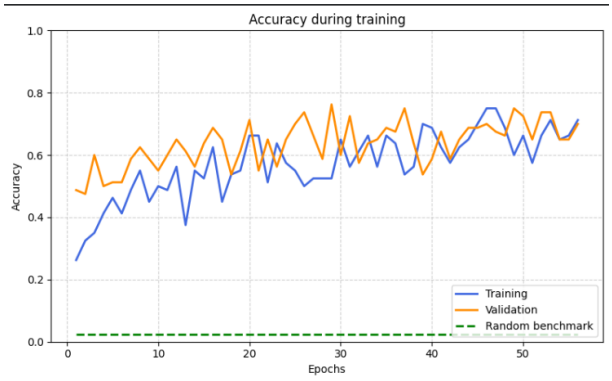


Fig. 6: CNN fine-tuning accuracy with `class_limit = 40`

of 40, there is a too small amount of samples with this high amount of frames for training the model. Having a too small amount of data, overfitting with the data available results. The validation accuracy remains around 40%. Having smaller sequence lengths (5 and 20), more videos can become part of the training resulting in more patterns that are also available in test data. A more general definition of the each classes traits results. The accuracies for sequence lengths 5 and 20 are both above 90 percent. Still, the model trained with videos consisting of 20 frames performs a bit better. This is probably because more frames contain more information used for classification.

Trying this experiment also with other values for `class_limit`, shows that the task becomes harder to solve (Fig. 8), even for the LSTM model, since more classes are eligible as a classification result. Also having more classes available, the top $k = 5$ classes with best accuracies can be plotted since the `CSVLogger` also captures this. Clearly, these classes have higher accuracies than the other ones. They have patterns that make them more easily separatable from other classes.

Selecting more classes for high sequence lengths does not solve the problem of too small amounts of data (Fig. 9). Loss increases since the problem of too small amounts of long videos persists over the all classes. Overfitting results:

Training the LSTM takes more time than finetuning the InceptionV3 model. Stacking two big models above each other creates a huge combined one. Also, two steps have to be performed: Feature extraction and actual training. The method is more complex and needs more time and computational resources.

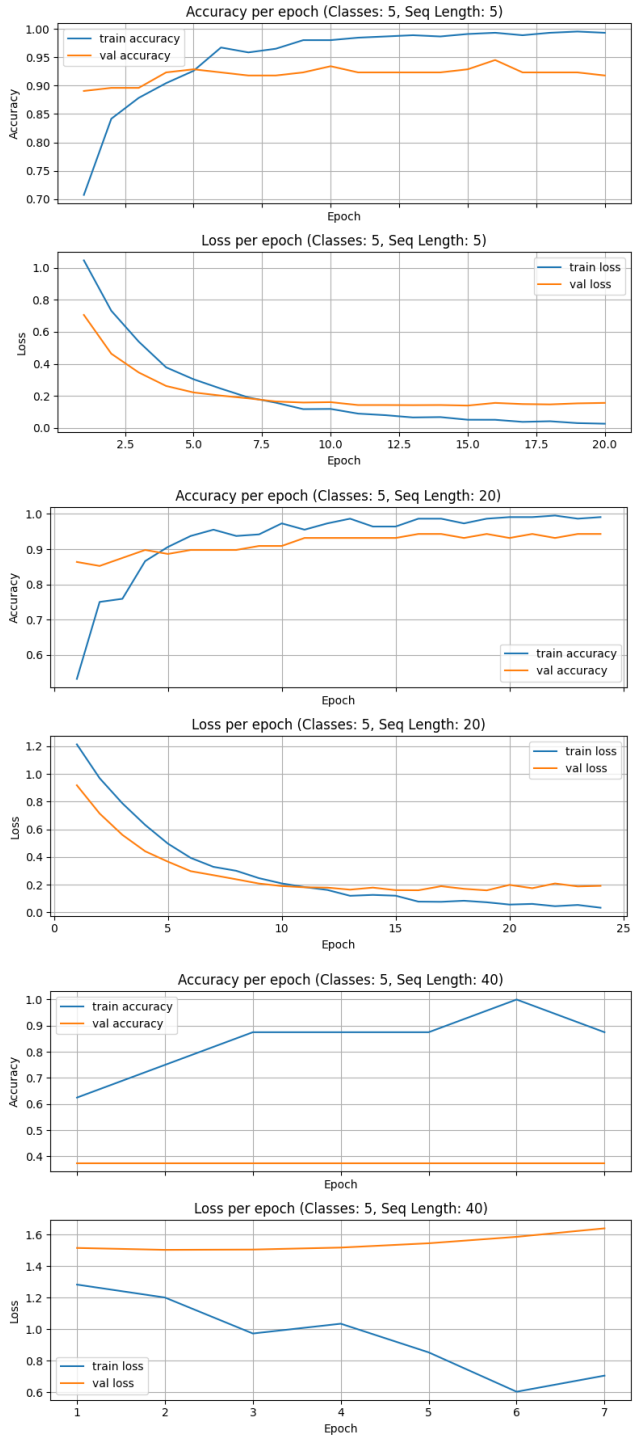


Fig. 7: LSTM trainings accuracy and loss with `class_limit=5` and seq length from 5 to 40

VI. CONCLUSIONS

Four methods have been used to solve the video classification. The first two approaches (random and fixed class assignment) deal as baselines for the classification task. The random prediction matched chance performance, and fixed prediction performance followed class distribution. These results provide a lower-bound benchmark for future models. Any meaningful classifier should surpass the best fixed baseline. During fine-tuning, a model is adapted to fit it to the new domain. Using an image classification model, it classifies images and not the entire video. An idea would be to implement result averaging through all frames belonging to one video to get an average class probability distribution for all frames. The CNN with LSTM implementation is the only one looking at all frames belonging to one video. The architecture is more complex than just using a CNN

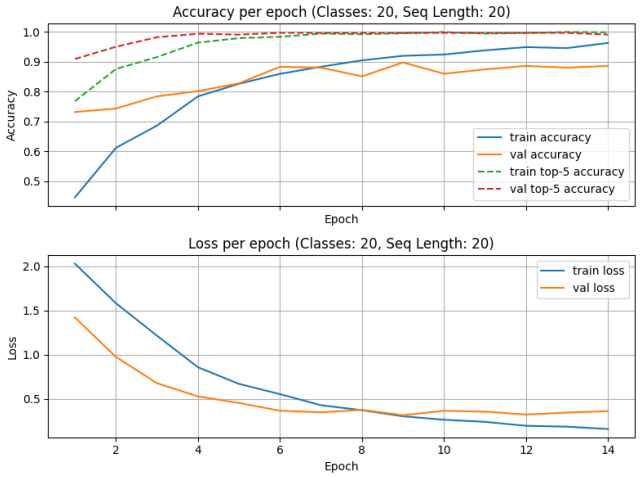


Fig. 8: LSTM trainings accuracy and loss with `class_limit=20` and `seq_length=20`

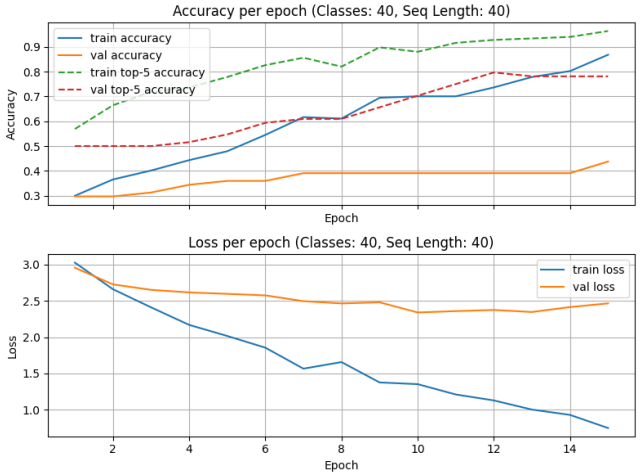


Fig. 9: LSTM trainings accuracy in the top 5 and in general and loss with `class_limit=40` and `seq_length=40`

model. Still, it delivers the best results when having a sufficient amount of videos belonging to each class. Further experiments could be performed using other hyperparameter values, model parameters or combinations of video classes.

VII. TIME LOG

Task	**Marcel (h)**	**Carlos (h)**
Programming – Task 1	1	4
Programming – Task 2	2	5
Programming – Task 3	6	2
Writing the Report	14	5
Total	**23**	**16**

TABLE II: Work Distribution Between Team Members

VIII. REFERENCES

REFERENCES

[1] K. Soomro, A. R. Zamir, and M. Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild," CRCV–University of Central Florida, 2012.

[2] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in *Proc. IEEE CVPR*, 2016.

[3] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, 2010.

[5] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Online]. Available: https://www.tensorflow.org/

[6] FFmpeg Developers, "FFmpeg tool," [Online]. Available: https://ffmpeg.org/